

Introducción

Dos formas de acceder a los datos de la flota: un **WebSocket** que transmite posiciones, estados y eventos en tiempo real, y una **API REST** para consultar el estado actual y datos históricos bajo demanda.

El servicio expone un canal **WebSocket** que envía datos en formato **JSON**. Apenas se establece la conexión, el servidor entrega un snapshot con la última posición conocida de cada equipo y, a partir de ahí, emite mensajes en vivo cada vez que llega una nueva posición, cambia el estado de un equipo o se genera un evento.

Cada conexión está asociada a un **usuario** mediante un token. El feed queda automáticamente acotado a los equipos que ese usuario tiene permiso de ver: no hace falta suscribirse ni filtrar del lado del cliente.

Conexión

Se conecta a la URL del WebSocket pasando el token como parámetro de query:

ENDPOINT

```
wss://SU-SERVIDOR/api/socket?token=EL_TOKEN
```

- Usar `wss://` (conexión cifrada). El host y el token se entregan por separado.
- El `token` identifica al usuario; el servidor valida y autoriza la conexión.
- No requiere cabeceras adicionales ni un login previo: la autenticación viaja en la URL.

💡 **Snapshot inicial.** Ni bien conecta, recibe un mensaje `{ "positions": [...] }` con la última posición de cada equipo. Luego comienza el flujo de actualizaciones.

Estructura de mensajes

Cada mensaje recibido es un **objeto JSON** que contiene una o más de las siguientes claves. En cada mensaje solo viajan las claves que tienen novedades — el resto se omite.

Clave	Tipo	Contenido
positions	array	Nuevas posiciones (lo más frecuente).
devices	array	Cambios de estado del equipo (online/offline, edición).
events	array	Eventos y alarmas generados en vivo.

EJEMPLO DE MENSAJE

```
{
  "devices":  [ { "id": 5, "status": "online", "lastUpdate": "2026-07-18T14:30:05.000+00:00" },
  "positions": [ { "deviceId": 5, "latitude": -34.6037, "longitude": -58.3816, "speed": 12.5 },
  "events":    [ { "type": "device0verspeed", "deviceId": 5, "eventTime": "2026-07-18T14:30:05.000+00:00" },
}
```

Objeto position

El elemento principal del feed. Los campos de arriba son fijos; todo lo variable (según el modelo de equipo) vive dentro de attributes .

Campo	Tipo	Descripción
id	number	Identificador único de la posición.
deviceId	number	Id del equipo al que pertenece.
protocol	string	Protocolo del equipo (ej. h02 , jt808).
deviceTime	datetime	Hora del equipo (ISO-8601 con zona horaria).
fixTime	datetime	Hora del posicionamiento GPS.
serverTime	datetime	Hora en que el servidor recibió el dato.
valid	boolean	true si el fix GPS es válido.
outdated	boolean	true si la posición reusa coordenadas previas (sin fix nuevo).
latitude	number	Latitud en grados decimales.
longitude	number	Longitud en grados decimales.
altitude	number	Altitud en metros.
speed	number	Velocidad en nudos. km/h = speed × 1,852.
course	number	Rumbo en grados (0–359).
address	string · null	Dirección geocodificada, si está disponible.
geofenceIds	array · null	Ids de geocercas que contienen la posición.
attributes	object	Datos adicionales variables (ver sección siguiente).

El objeto `attributes`

Contenedor dinámico donde el equipo reporta telemetría adicional. **Qué claves aparecen depende del modelo y del protocolo** — no todas están siempre presentes. Las más habituales:

Clave	Tipo	Descripción
<code>ignition</code>	<code>boolean</code>	Estado de la ignición (motor encendido).
<code>motion</code>	<code>boolean</code>	<code>true</code> si el equipo está en movimiento.
<code>totalDistance</code>	<code>number</code>	Distancia acumulada calculada por el servidor, en metros .
<code>odometer</code>	<code>number</code>	Odómetro reportado por el hardware, en metros.
<code>distance</code>	<code>number</code>	Distancia desde la posición anterior, en metros.
<code>hours</code>	<code>number</code>	Horas de motor acumuladas, en milisegundos.
<code>battery</code>	<code>number</code>	Voltaje de la batería.
<code>batteryLevel</code>	<code>number</code>	Nivel de batería en porcentaje.
<code>power</code>	<code>number</code>	Voltaje de alimentación externa.
<code>sat</code>	<code>number</code>	Cantidad de satélites usados en el fix.
<code>rsi</code>	<code>number</code>	Nivel de señal celular.
<code>blocked</code>	<code>boolean</code>	Estado del corte de motor, si el equipo lo soporta.
<code>alarm</code>	<code>string</code>	Presente solo si hay alarma (ej. <code>sos</code> , <code>powerCut</code>).

Objeto `device`

Llega en la clave `devices` cuando un equipo cambia de estado (se conecta o desconecta) o se modifica su ficha.

Campo	Tipo	Descripción
<code>id</code>	<code>number</code>	Identificador del equipo.
<code>name</code>	<code>string</code>	Nombre del equipo (habitualmente la patente).
<code>uniqueId</code>	<code>string</code>	Identificador físico (IMEI / serial).
<code>status</code>	<code>string</code>	<code>online</code> <code>offline</code> o <code>unknown</code> .
<code>lastUpdate</code>	<code>datetime</code>	Última comunicación recibida.
<code>positionId</code>	<code>number</code>	Id de la última posición del equipo.
<code>groupId</code>	<code>number</code>	Grupo al que pertenece (0 si ninguno).
<code>phone</code>	<code>string</code>	Teléfono de la SIM, si está cargado.
<code>model</code>	<code>string</code>	Modelo del equipo.
<code>disabled</code>	<code>boolean</code>	<code>true</code> si el equipo está deshabilitado.

Objeto `event`

Llega en la clave `events` . Solo se reciben eventos generados **mientras la conexión está activa** (no históricos).

Campo	Tipo	Descripción
id	number	Identificador del evento.
type	string	Tipo de evento (ver lista abajo).
eventTime	datetime	Momento del evento.
deviceId	number	Equipo asociado.
positionId	number	Posición vinculada al evento.
geofenceId	number	Geocerca involucrada (0 si no aplica).
attributes	object	Datos del evento (ej. alarm , speedLimit).

Tipos de evento frecuentes

- deviceOnline / deviceOffline — conexión / desconexión del equipo.
- ignitionOn / ignitionOff — encendido / apagado del motor.
- deviceMoving / deviceStopped — inicio / fin de movimiento.
- deviceOverspeed — exceso de velocidad (incluye speedLimit en attributes).
- geofenceEnter / geofenceExit — entrada / salida de geocerca.
- alarm — alarma; el tipo específico va en attributes.alarm (ej. sos).

Reglas de parseo

- **Iterar solo las claves presentes.** Nunca asumir que un mensaje trae las tres; validar cada una antes de procesar.
- **Cada valor es un array**, aunque contenga un solo elemento.
- **La telemetría variable vive en attributes** . Los campos de nivel superior son fijos; el resto es dinámico según el protocolo.
- **Velocidad en nudos** → convertir con $\text{km/h} = \text{speed} \times 1,852$.
- **Distancias en metros, horas de motor en milisegundos, fechas en ISO-8601 con zona horaria.**
- **Reconexión.** El servidor cierra la conexión por inactividad; el cliente debe reconectar automáticamente.

⚠ **Vencimiento del token.** Si el token expira o se revoca, la conexión se cae y los reintentos fallarán hasta renovarlo. Contemplar esta situación en la lógica de reconexión.

Ejemplos de código

Node.js

JAVASCRIPT

```
const WebSocket = require('ws');

const HOST = 'SU-SERVIDOR';
const TOKEN = 'EL_TOKEN';

function connect() {
  const ws = new WebSocket(`wss://${HOST}/api/socket?token=${encodeURIComponent(TOKEN)}`);

  ws.on('open', () => console.log('conectado'));

  ws.on('message', (raw) => {
    const msg = JSON.parse(raw);

    for (const p of msg.positions ?? []) {
      const kmh = Math.round(p.speed * 1.852);
      console.log(`device ${p.deviceId}: ${p.latitude},${p.longitude} - ${kmh} km/h`);
    }
    for (const e of msg.events ?? []) {
      console.log(`evento ${e.type} (device ${e.deviceId})`);
    }
    for (const d of msg.devices ?? []) {
      console.log(`device ${d.id} → ${d.status}`);
    }
  });

  ws.on('close', () => setTimeout(connect, 5000)); // reconexión
  ws.on('error', (err) => console.error(err.message));
}

connect();
```

Python

PYTHON

```
# pip install websocket-client
import json, websocket

HOST, TOKEN = "SU-SERVIDOR", "EL_TOKEN"

def on_message(ws, raw):
    msg = json.loads(raw)
    for p in msg.get("positions", []):
        kmh = round(p["speed"] * 1.852)
        print(f'device {p["deviceId"]}: {p["latitude"]},{p["longitude"]} - {kmh} km/h')
    for e in msg.get("events", []):
        print(f'evento {e["type"]} (device {e["deviceId"]})')

websocket.WebSocketApp(
    f"wss://{HOST}/api/socket?token={TOKEN}",
    on_message=on_message,
).run_forever(reconnect=5)
```

API REST — Autenticación

Además del feed en tiempo real, la API REST permite hacer consultas puntuales. Todas las respuestas son en formato **JSON** y quedan acotadas a los equipos que el usuario tiene permiso de ver.

- **URL base:** `https://SU-SERVIDOR/api`
- Este documento cubre únicamente las operaciones de lectura (GET).

Cada solicitud se autentica con la cabecera `Authorization` , en **cualquiera de estas dos formas**:

Opción 1 • Token Bearer

Recomendada para integraciones: no expone la contraseña del usuario y puede revocarse.

```
AUTHORIZATION: BEARER
```

```
curl -H "Authorization: Bearer EL_TOKEN" \
  "https://SU-SERVIDOR/api/devices"
```

Opción 2 • Usuario y contraseña Basic

Autenticación HTTP Basic estándar con el **email y la contraseña** del usuario (la cabecera lleva `email:contraseña` codificado en Base64; `curl -u` lo hace solo).

```
AUTHORIZATION: BASIC
```

```
curl -u "usuario@email.com:CONTRASEÑA" \
  "https://SU-SERVIDOR/api/devices"
```

💡 La autenticación **Basic** aplica solo a la API REST. El [WebSocket](#) se autentica con el **token en la URL**, como se muestra en la sección de conexión.

GET /api/devices

Devuelve la lista de equipos del usuario. Cada elemento tiene la estructura del [objeto device](#) . Sin parámetros, lista todos los equipos accesibles.

Parámetro	Tipo	Descripción
id	number · repetible	Uno o varios equipos por id (? id=5&id=8).
uniqueId	string · repetible	Por identificador físico (IMEI / serial).
keyword	string	Búsqueda por nombre, uniqueId, teléfono o modelo.
limit	number	Máximo de resultados (paginación).
offset	number	Desplazamiento inicial (paginación).

EJEMPLO

```
curl -H "Authorization: Bearer EL_TOKEN" \
  "https://SU-SERVIDOR/api/devices?keyword=AB123CD"
```

GET /api/positions

Devuelve posiciones con la estructura del [objeto position](#) . Su comportamiento depende de los parámetros:

Parámetros	Resultado
(ninguno)	Última posición de cada equipo del usuario.
deviceId	Última posición de ese equipo.
deviceId + from + to	Histórico de posiciones del equipo en el rango.
id	Posiciones puntuales por id (repetible: ? id=100&id=101).

Parámetro	Tipo	Descripción
deviceId	<code>number</code>	Equipo a consultar.
from	<code>datetime</code>	Inicio del rango, ISO-8601 (ej. 2026-07-18T00:00:00Z).
to	<code>datetime</code>	Fin del rango, ISO-8601.
id	<code>number · repetible</code>	Posiciones específicas por id.

EJEMPLO · HISTÓRICO

```
curl -H "Authorization: Bearer EL_TOKEN" \
  "https://SU-SERVIDOR/api/positions?deviceId=5&from=2026-07-18T00:00:00Z&to=2026-07-18T00:00:00Z"
```

 **Rango de fechas.** El histórico puede tener un límite máximo de período y de cantidad de puntos por consulta, según la configuración del servidor.

GET /api/**events**/**{id}**

Devuelve un [evento](#) puntual por su identificador.

EJEMPLO

```
curl -H "Authorization: Bearer EL_TOKEN" \
  "https://SU-SERVIDOR/api/events/90012"
```

 **Eventos por rango.** Para obtener todos los eventos de un equipo en un período, usar el reporte de eventos: GET /api/reports/events?deviceId=&from=&to=&type=

GET /api/geofences

Devuelve la lista de geocercas accesibles. Sin parámetros, lista todas las del usuario.

Parámetro	Tipo	Descripción
deviceId	number	Geocercas asignadas a un equipo.
groupId	number	Geocercas de un grupo.
userId	number	Geocercas de un usuario (requiere permiso).
limit / offset	number	Paginación.

Estructura del objeto geofence

Campo	Tipo	Descripción
id	number	Identificador de la geocerca.
name	string	Nombre.
description	string · null	Descripción opcional.
area	string	Geometría en formato WKT — POLYGON , CIRCLE o LINESTRING con coordenadas lat/lon.
calendarId	number	Calendario asociado (0 si ninguno).
attributes	object	Datos adicionales (color, velocidad máxima, etc.).

EJEMPLO

```
curl -H "Authorization: Bearer EL_TOKEN" \
  "https://SU-SERVIDOR/api/geofences?deviceId=5"
```

